

Greedy Questions

Question 1 :

Maximum Balanced String Partitions

We have balanced string `str` of size `N` with an equal number of `L` and `R`, the task is to find a maximum number `X`, such that a given string can be partitioned into `X` balanced substring. A string is called to be balanced if the number of '`L`'s in the string equals the number of '`R`'s.

Input : "LRRRRLRLRLRL"

Output : 3

Question 2 :

Kth largest odd number in a given range

We have two variables `L` and `R`, indicating a range of integers from `L` to `R` inclusive, and a number `K`, the task is to find `K`th largest odd number. If `K > number of odd numbers in the range L to R` then return 0.

Sample Input 1 : `L = -3, R = 3, K = 1`

Sample Output 1 : 3

Question 3 :

Lexicographically smallest string of length N and sum K

We have two integers `N` and `K`. The task is to print the lexicographically smallest string of length `N` consisting of lower-case English alphabets such that the sum of the characters of the string equals to `K` where '`a`' = 1, '`b`' = 2, '`c`' = 3, and '`z`' = 26.

Sample Input 1 : `N = 5, K = 42`

Sample Output 1 : `aamz`

Sample Input 2 : `N = 3, K = 25`

Sample Output 2 : `aaw`

Question 4 :

Best Time to Buy and Sell Stock

Given an array `prices[]` of length `N`, representing the prices of the stocks on different days, the task is to find the maximum profit possible for buying and selling the stocks on different days using transactions where at most one transaction is allowed.

Note: Stock must be bought before being sold.

Sample Input 1: `prices[] = {7, 6, 4, 3, 1}`

Sample Output 1: 0

Sample Input 2: `prices[] = {7, 1, 5, 3, 6, 4}`

Sample Output 2: 5

Question 5 :

Split the given array into K sub-arrays

We have an `Array[]` of N elements and a number K . ($1 \leq K \leq N$). Split the given array into K subarrays (they must cover all the elements). The maximum subarray sum achievable out of K subarrays formed must be the minimum possible. Find that possible subarray sum.

Sample Input 1 : `Array[] = {1, 1, 2}` $K = 2$

Sample Output 1 : 2

Sample Input 2 : `Array[] = {1, 2, 3, 4}`, $K = 3$

Sample Output 2 : 4

Greedy Solutions

Solution 1:

Time Complexity : $O(n)$

Space Complexity: $O(1)$

```
import java.util.*;
```

```
class Solution {
```

```
static int BalancedPartition(String str, int n){
```

```
    if (n == 0)
        return 0;
```

```
    int r = 0, l = 0;
```

```
    int ans = 0;
```

```
    for(int i = 0; i < n; i++) {
```

```
        if (str.charAt(i) == 'R'){
```

```
            r++;
```

```
        }
```

```
        else if (str.charAt(i) == 'L'){
```

```
            l++;
```

```
        }
```

```
        if (r == l){
```

```
            ans++;
```

```
        }
```

```
    }
```

```
    return ans;
```

```
}
```

```
public static void main(String[] args){
```

```
    String str = "LLRRRLRLRL";
```

```
    int n = str.length();
```

```
    System.out.print(BalancedPartition(str, n) + "\n");
```

```
}
```

```
}
```

Solution 2 :

Time Complexity : $O(1)$

Space Complexity: $O(1)$

```
class Solution {  
  
    public static int kthOdd(int[] range, int K) {  
  
        if (K <= 0)  
            return 0;  
  
        int L = range[0];  
        int R = range[1];  
  
        if ((R & 1) > 0) {  
            int Count = (int) Math.ceil((R - L + 1) / 2);  
            if (K > Count)  
                return 0;  
            else  
                return (R - 2 * K + 2);  
        } else {  
            int Count = (R - L + 1) / 2;  
            if (K > Count)  
                return 0;  
            else  
                return (R - 2 * K + 1);  
        }  
    }  
  
    public static void main(String args[]){  
        int[] p = { -10, 10 };  
        int k = 8;  
        System.out.println(kthOdd(p, k));  
    }  
}
```

Solution 3 :

Time Complexity : $O(n)$

Space Complexity: $O(n)$

```
import java.util.Arrays;
```

```
public class Main {
```

```
    public static char[] lexo_small(int n, int k){
```

```
        char arr[] = new char[n];
```

```
        Arrays.fill(arr, 'a');
```

```
        for (int i = n - 1; i >= 0; i--) {
```

```
            k -= i;
```

```
            if (k >= 0) {
```

```
                if (k >= 26) {
```

```
                    arr[i] = 'z';
```

```
                    k -= 26;
```

```
                }
```

```
            } else {
```

```
                arr[i] = (char)(k + 97 - 1);
```

```
                k -= arr[i] - 'a' + 1;
```

```
            }
```

```
        } else
```

```
            break;
```

```
        k += i;
```

```
    }
```

```
    return arr;
```

```
}
```

```
public static void main(String[] args){
```

```
    int n = 5, k = 42;
```

```
    char arr[] = lexo_small(n, k);
```

```
    System.out.print(new String(arr));
```

```
}
```

```
}
```

```

g args[] {
    6, 4 };
Profit(prices, n);

```

Space Complexity: $O(1)$

```
public static void main(String args[]){
    int prices[] = { 7, 1, 5, 6, 4 };
    int n = prices.length;
    int max_profit = maxProfit(prices, n);
    System.out.println(max_profit);
}
```

(Here 'c' here depicts combinations i.e. $\frac{(n-1)!}{((n-k)!*(k-1)!}$ Where N is the number of elements of the array and K is the number of divisions that we are having)

[illegible]

```
        if (k == 1) {
            maxsum = Math.max(maxsum, sum);
            sum = 0;
            for (int i = index; i < n; i++) {
                sum += a[i];
            }
            maxsum = Math.max(maxsum, sum);
            ans = Math.min(ans, maxsum);
            return;
        }
        sum = 0;
        for (int i = index; i < n; i++) {
            sum += a[i];
            maxsum = Math.max(maxsum, sum);
            solve(a, n, k - 1, i + 1, sum, maxsum);
        }
    }

    public static void main(String[] args) {
        int arr[] = { 1, 2, 3, 4 };
        int k = 3; // K divisions
        int n = 4; // Size of Array
        solve(arr, n, k, 0, 0, 0);
        System.out.println(ans + "\n");
    }
}
```